FlowMiner: A Powerful Model Based on Flow Correlation Mining for Encrypted Traffic Classification

Hongbo Xu[1,2,4,*], Chengxiang Si[3,*], Shuhao Li[4,†], Zhenyu Cheng[4],
Chenxu Wang[1,2], Jiang Xie[4], Peishuai Sun[1,2,4], Qingyun Liu[1,2]
[1]Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
[2]School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
[3]National Computer Network Emergency Response Technical Team/Coordination Center of China, Beijing, China
[4]Zhongguancun Laboratory, Beijing, China

IEEE INFOCOM 2025 - IEEE Conference on Computer Communications | 979-8-3315-4305-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/INFOCOM55648.2025.11044724

***Abstract*—With the continuous development of traffic encryption techniques, the encrypted traffic classification task faces increasing challenges. Existing encrypted traffic classification methods primarily address the encryption problem by extracting side channel features, such as packet lengths and timing sequences. However, existing methods usually classify traffic at the flow or packet level. There are generally no mechanisms for mining correlations between different flow samples. Because of the loss of correlation information, the performance of these models in multiple tasks is greatly affected.**

To solve the above problems, we propose FlowMiner, a traffic graph classification model that mines correlations among different flow samples. It first extracts the temporal, length, content, byte distribution, and crossover features. FlowMiner uses these features as initial node features. Then, it constructs flow interaction graphs by analyzing the association between different flows. FlowMiner utilizes Graph Neural Networks to extract association features between different flows. It then generates a robust graph-level representation vector through an integrated pooling module, which is subsequently used for classification. Extensive experimental results on eight datasets show that FlowMiner significantly outperforms multiple state-of-the-art methods. Additionally, results from two real-world evaluation scenarios show that FlowMiner achieves over 95% precision in identifying malicious traffic, proving its effectiveness for practical applications.

***Index Terms*—Network Traffic Classification, Encrypted Traffic, Graph Neural Networks, Graph Classification**

I. Introduction

Network traffic classification is one of the fundamental tasks in network security and network management. It is widely used in application classification, intrusion detection, and Web service identification tasks. In recent years, due to the wide application of encryption technology (e.g., TLS 1.3), traditional DPI technology no longer applies to encrypted traffic classification tasks. Meanwhile, the rapid development of traffic camouflage and obfuscation techniques has brought significant challenges to traffic classification. Attackers and cybercriminals can utilize technologies such as Virtual Private Networks (VPN), censorship-resistant proxy, and The Onion Router (Tor) to evade censorship systems. Most encrypted traffic classification techniques rely on packet length or timing information to extract flow or packet-level statistical features. Then, utilizing statistical or other semantic features, these methods use machine or deep learning techniques to learn the traffic representation. However, these methods still have some problems.

(1) Existing traffic classification methods are limited because they lack comprehensive feature extraction, such as only extracting spatio-temporal traffic features [1] or only byte features [2], [3]. Therefore, most of these methods are adequate for a particular task and cannot adapt well to different traffic classification tasks.

(2) Statistical traffic features are represented as tabular, heterogeneous data. For example, there is no obvious correlation between temporal and length statistical features. Since deep learning techniques have yet to show significant advantages in tabular data [4], how to combine heterogeneous features of different dimensions is still a problem worth exploring.

(3) Existing traffic classification techniques are mostly based on the flow or packet level. Due to detecting at one level and not mining the correlation relationship between different flows, these methods lose a lot of correlation information. The Graph Neural Network (GNN) has received attention from many scholars due to its powerful ability to model association relationships [5]. Researchers have used GNNs for specific traffic classification tasks, such as application classification [6], [7]. **However, these methods only utilize partial information from the traffic, such as bytes and packet lengths, for graph construction, without fully exploiting the correlations between different flows.** How to leverage GNNs to learn the intrinsic associations in traffic and construct a generalized traffic classification model still requires further exploration.

To address the above issues, we propose FlowMiner, a novel encrypted traffic classification model with deeply integrated multidimensional heterogeneous features such as temporal, content, and behavioral dimensions. (1) Specifically, we propose a novel method for constructing and learning representations of traffic graphs that capture flow interaction behaviors.

* These authors contributed equally to this work. † Shuhao Li is the corresponding author; his email is lishuhao@zgclab.edu.cn.

Distinguishing itself from prior work that constructs traffic graphs at the level of single flows or packet, FlowMiner's essential contribution lies in achieving traffic classification at the level of inter-flow interaction graphs, thereby expanding the scope of exploitable information in encrypted traffic. (2) Furthermore, we propose new traffic features. **Unlike previous methods focused solely on raw byte content representation**, we generate multidimensional statistical features for traffic bytes. The cross-features represent traffic content **from a novel dimension, considering correlations between different attribute columns of traffic features** (e.g., correlations between packet length and time features), thereby revealing inter-column relationships of traffic features. (3) Simultaneously, we meticulously design FlowMiner's GNN representation learning component and propose an integrated decision pooling module. **A key innovation of this module is introducing a decision mechanism during graph pooling, which balances the consideration of information across all nodes in the graph rather than simply taking the maximum or average values.** This enhancement aims to better capture the overall features of flow interaction graphs.

The contributions of this paper are summarized as follows:

(1) We introduce **innovative features from a new perspective: byte statistical distribution and nonlinear crossover features**. These enhanced features supplement both byte pattern learning and the learning of crossover relationships between different features, addressing gaps in existing methods.

(2) We propose FlowMiner, a generic encrypted traffic classification model. **FlowMiner constructs flow interaction graphs by mining correlations between different flow samples.** Then, FlowMiner uses a well-designed graph neural network to learn shallow statistical features and deep behavioral interaction features from flow interaction graphs. This approach fosters a beneficial complementarity between heterogeneous statistical and behavioral features, enhancing the model's predictive capabilities. By constructing robust graph representation vectors, FlowMiner shows broad applicability to multiple traffic classification tasks.

(3) Extensive experimental results **on eight traffic datasets** show that FlowMiner exhibits strong traffic representation learning capability and **more robust performance** than existing state-of-the-art (SOTA) methods (**improving the average F1 score by 10.18% compared to the best baseline model, BIND, across all tasks.**). In addition, FlowMiner maintains a tiny model size while still achieving optimal classification performance, outperforming existing deep learning methods. **The results from real-world deployment tests indicate that our model can rival the cutting-edge intrusion detection ruleset, Emerging Threat Ruleset, without requiring any expert knowledge (achieving a precision rate of over 95%)**.

II. Motivation

There are currently three encrypted traffic classification levels: packet, flow, and host.

Packet-level traffic classification typically focuses on the properties of individual packets. These features include packet size, arrival time sequence, and header field distribution. However, due to traffic content encryption, limited traffic information is obtained at the packet level.

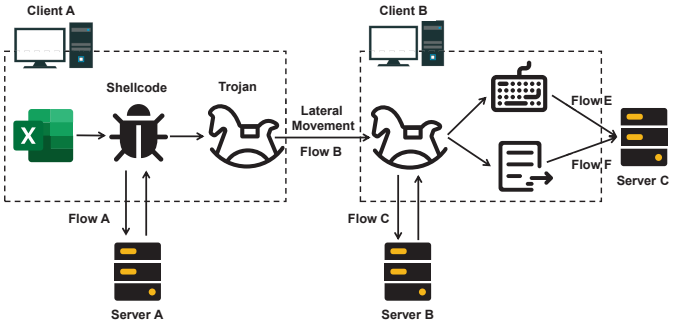

Fig. 1: Downloader Trojan Attack Illustration.

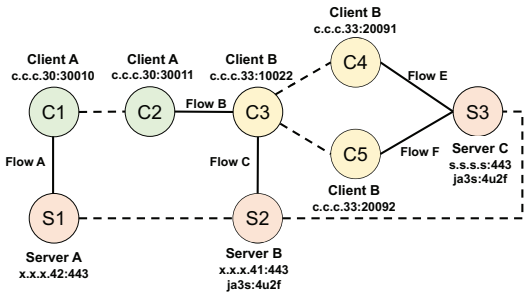

Fig. 2: An Example of Building a Flow Interaction Graph.

Flow-level traffic classification is usually based on a sequence of associated packets. A flow is identified as a single communication session characterized by a network quintuple: source IP address, destination IP address, source port, destination port, and protocol type. Classification methods at this level usually consider the overall features of the flow. These features include the distribution of packet size and arrival time, the duration of the flow, and the total amount of data transferred.

Host-level traffic classification methods usually focus on a single host's network behavior and communication patterns. Different hosts may exhibit some features, such as frequency of communication. However, host-level traffic classification is often challenged by the presence of more noisy information.

However, these traffic analysis tasks still have limitations. As shown in Figure 1, the scenario shows the attack process of a downloader Trojan. First, the user receives a Trojan document. After the malicious document is executed on client A, the downloader Trojan conducts the theft process in three communication stages. (1) In the first stage, the module downloader is retrieved from server A using the HTTP protocol. The Trojan file is downloaded, and lateral movement is performed. At this time, the Trojan is copied to client B. (2) In the second stage, the Trojan on client B obtains multiple malware plugins from server B through the TLS protocol. (3) In the third stage, each plugin automatically encrypts the stolen data and uploads it to server C through the TLS protocol.

We found that packet-level, flow-level, and host-level anal-

yses can only analyze a single behavioral interaction in this scenario and cannot model multiple behaviors of different flow samples between other hosts (for example, Flow A and C).

To solve the above problem, we propose FlowMiner, a novel traffic classification model equipped with a cross-flow interaction analysis mechanism. FlowMiner aims to construct the above scenario into a flow interaction graph and classify traffic at the traffic graph level. Figure 2 illustrates how multiple flows on different hosts are interconnected through IP address associations and traffic information. Initially, a flow, such as Flow A, is divided into two distinct nodes: C1 and S1. Solid lines indicate actual traffic interactions, establishing an edge. Dashed lines represent connections based on flow association relationships. For instance, nodes S1 and S2 are associated with similar Class C IP addresses. Nodes S2 and S3 are associated with identical JA3S fingerprints, suggesting that the servers exhibit similar behaviors during the TLS handshake process. FlowMiner leverages a GNN model to perform comprehensive flow data analysis, learning the associations between flow samples and further enhancing its ability to learn traffic representations.

III. METHODOLOGY

In this paper, a **Flow Interaction Graph** is defined as a graph that captures the relationships and interactions among multiple network flows. As shown in Figure 3, FlowMiner contains two main parts: constructing flow interaction graphs and learning their representations.

A. Node Construction Mechanism

A flow characterizes the transmission behavior of an application between a client and a server in network communication. Therefore, it is easy to think of one application as a node. For example, we can identify a node by IP address and port number. However, this method will result in feature conflicts on the same node due to port multiplexing, as even identical five-tuples may be classified into different flows because of the presence of long connections. Moreover, the same application may have multiple transfer behaviors. To solve this problem, we uniquely identify a node with a triple (flow ID, IP address, port number), enabling us to split one flow into two nodes. At the same time, we can analyze the correlation between multiple nodes using IP and port information.

B. Flow List Partitioning

For a raw traffic file, we first extract it into a list of flows with various flow features. To ensure the stability of graph learning, we divide the flow list into multiple graphs. We introduce a concept of **maximum expectation node number** to balance the number of nodes in each graph structure. The concept can be formally described as follows. Given a flow list of length N, we set max_nodes as the expected maximum number of nodes in each subgraph to ensure both computational and memory demands are controlled while maintaining a moderate graph scale. The original flow list is divided into k sublists, where

$$k = \left\lfloor \frac{N}{\text{max\_nodes}} \right\rfloor + I(N \mod \text{max\_nodes} > 0) \quad (1)$$

Here, I is an indicator function, being one if the condition in brackets is true and zero otherwise. The first $k-1$ sublists each contain precisely max_nodes flows, while the kth sublist accommodates all remaining flows. This strategy ensures that the number of nodes in each generated subgraph is balanced for subsequent training, avoiding situations where extreme numbers of nodes are delineated. Each sub-list is used to construct a graph for subsequent computations and analyses.

C. Basic Node Feature Extraction

We first split each flow into two nodes using triplet information (flow ID, IP address, and port number). Then, we assign the transport features to the server and client nodes, utilizing the feature information within the same flow. Therefore, the flow transmission feature of each node consists of two types: one is the transmission feature from itself to the outside, and the other is the bidirectional transmission feature in the flow. The primary node features of each node can be specifically categorized into four types: temporal features, packet length features, traffic content features, and our proposed byte distribution features. The first three types of features are built-in features within the NFStream library.

Byte distribution features are a series of features built on payload bytes. This feature series consists of four parts. The first part is the feature based on byte content distribution. It includes the occurrence count of each byte in the 0-255 range. The second part is payload length distribution features, including the payload length count in each packet. The two parts of the feature are used as raw distribution features without statistical value calculation and can be used to extract higher-order features from neural networks automatically.

The third and fourth parts are features constructed based on the number of printable characters and the popcount value, respectively. In particular, printable characters are characters in the ASCII range (e.g., letters, numbers, and punctuation marks). This feature can be used to differentiate between different traffic protocols. This is because the distribution of printable characters can significantly vary between different protocols. On the other hand, the popcount value, i.e., the number of '1' in a segment of a binary string, can be used to characterize the entropy value of the entire packet content, which also can be used to discriminate traffic for the presence of the full encryption feature [8]. Fully encrypted traffic typically exhibits a higher entropy value in its packet content because the encryption process produces uniformly distributed binary strings. These two features provide the model with intuitive information about network traffic content and encryption state and enhance our model's discriminative power on different traffic classification tasks.

To conserve computational complexity, for byte features, we extract only the first six bytes of information from the

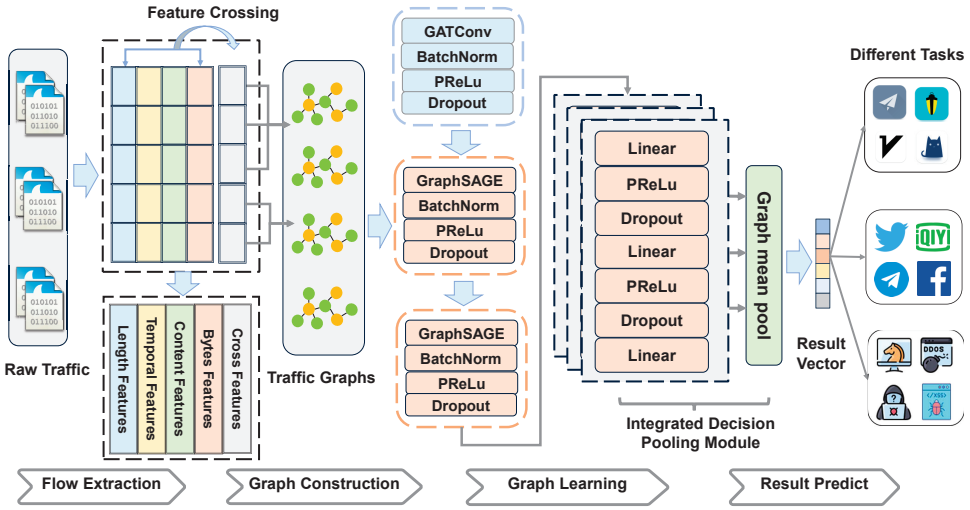

Fig. 3: FlowMiner Model Architecture.

payload of each packet within a flow. Different features for each node are labeled bidirectional and single, respectively. We compute the statistical features of each essential flow feature, including maximum, minimum, mean, median, mode, standard deviation, variance, range, quartiles (Q1, Q2, Q3), percentiles (P25, P50, P75, P90, P95), skewness, and kurtosis. **Further details on the features are available in our code implementation**[1].

D. Nonlinear Cross-feature Extraction

Feature crossing is commonly used in classification and regression tasks to capture complex relationships between features. To enhance the model's ability to capture complex patterns, we designed the cross-feature extraction module to capture higher-order relationships between heterogeneous flow features. This module includes various methods for implementing feature interactions, as follows.

Variability-Based Feature Selection: Initially, we prioritize features based on the standard deviation of their variability within the data and select ten features. This ensures that features with higher variability and potentially more information are selected for interaction.

Binning Crosses: We employ binning to convert the numerical feature f into categorical feature B_f through $B_f = \text{Bin}(f)$. We then form pairwise interactions, C_{f_1,f_2}, by concatenating the string representations of binned features, denoted as: $C_{f_1,f_2} = B_{f_1} \cdot B_{f_2}$.

Arithmetic Crosses: Utilizing arithmetic operations, namely multiplicative, additive, and subtractive operations, we conduct feature interactions between features f_1 and f_2 as: $f_1 \times f_2, f_1 + f_2, f_1 - f_2$, where M, A, and S represent multiplicative, additive, and subtractive interactions, respectively.

Statistical Crosses: The module integrates statistical interactions by computing pairwise correlations between selected features within each graph instance. Given a graph id g, the correlation between features f_1 and f_2 is: $\text{Corr}_g(f_1, f_2) = \text{corr}(f_1^{(g)}, f_2^{(g)})$, where $f_1^{(g)}$ and $f_2^{(g)}$ represent the feature values within graph g.

This feature crossing module generates non-linear high-order traffic features, offering a rich composite input feature space, which is vital for subsequent graph learning tasks. With the help of carefully designed interactive features and other node features, we can expand our model's input space in depth and breadth for various tasks. This elevates FlowMiner's representational capability.

E. Flow Interaction Graph Construction

We use several mechanisms to construct edges in the flow interaction graph to characterize the relationship between different flows better. The details are shown below.

(1) Constructing Edges Based on Actual Traffic Connections. We construct an undirected edge between a server node and a client node split from the same flow. This indicates an interactive behavioral association between the two nodes.

(2) Constructing Edges Based on Node Information Correlation. We observed that different server IPs of the same application or different proxy server IPs from the same proxy service provider exhibit the phenomenon of sharing a C-class subnet. Therefore, we filter the nodes with the same IP or IP C-class subnet to create the corresponding correlation list.

(3) Constructing Edges Based on Flow Information Correlation. First, we distinguish server-side and client-side nodes based on port number. The port number on the client side is typically larger than the port number on the server side. Then, we filter the corresponding relevance list based on the flow's content information. For example, nodes with Server Name Indication (SNI) that refer to google.com will be added to the same relevance list. Since SNI is a server-side attribute, the correlation list will only contain server nodes with the same SNI. The content information used in this paper is extracted from the NFStream library, which includes client fingerprint, TLS Server Name Indication, Host Name in the HTTP protocol, Query Name information in the DNS protocol, application types, etc. The application type is parsed by the

[1]More details can be found at https://github.com/MrRobotsAA/FlowMiner.

nDPI tool built into the NFStream library. For more details, please refer to our code implementation[1].

Let us assume a set P representing all packets in network traffic, with our objective to construct a graph $G = (V, E)$, where V signifies the set of vertices, representing packets with identical content information, and E is a set of edges, connecting nodes with similar content information and arranged by temporal order. Specifically, the process can be delineated as follows:

Content Information Filtering: For each packet p_i in P, it is assigned to corresponding relevance lists L_c based on specific content information C. That is,

$$L_c = \{p_i \in P | C(p_i) = c\}, \quad \forall c \in C \tag{2}$$

where c designates a particular value within content information C.

Timestamp Sorting: For every relevance list L_c it is ordered according to the flow start timestamp $T(p_i)$ within the packet. Formally:

$$L'_c = \text{sort}(L_c, key = T(p_i)) \tag{3}$$

Edge Construction: Subsequently, within the list L'_c, every pair of adjacent nodes is connected, meaning an edge is created for every pair of consecutive packets p_i, p_{i+1} in the list. We can define the set of edges E_c as:

$$E_c = \{(p_i, p_{i+1}) | p_i, p_{i+1} \in L'_c, i \in [1, |L'_c| - 1]\} \tag{4}$$

The total set of edges E is the union of all E_c. Through these three stages, we initiate from the original packet set P, eventually crafting a graph G. This approach preserves connection behaviors' correlation while reducing graph density. Meanwhile, it prevents the generated graph from having too high a degree of homogeneity. Note that all edges are undirected and assigned the same connection weight.

F. Model Architecture

FlowMiner's graph learning consists of several GNN modules, including the GraphSAGE (Graph Sample and Aggregative) and GATConv (Graph Attention Convolution) layers. As shown in Figure 3, we first extract flow lists from the original traffic data containing rich features. Then, we build a flow interaction graph based on the relationships between the flows and conduct feature learning through the various GNN layers mentioned above. Throughout the learning process, each stage incorporates batch normalization, PReLu [9] activation functions, and dropout layers to enhance the model's learning ability. Ultimately, the integrated decision pooling module generates individual prediction results for each node, while a graph-level result vector is produced through global pooling.

1) GATConv Layer: The GATConv layer enables the model to sample neighboring nodes by assigning different attention weights to them. The output of each node i in the graph at the attention layer is:

$$h'_i = \text{ReLU}\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{\Theta} h_j\right) \tag{5}$$

where the attention coefficients α_{ij} are computed as:

$$\alpha_{ij} = \frac{\exp\left(\text{LeakyReLU}\left(\mathbf{a}^\top [\mathbf{\Theta} h_i \,||\, \mathbf{\Theta} h_j]\right)\right)}{\sum_{k \in \mathcal{N}(i) \cup \{i\}} \exp\left(\text{LeakyReLU}\left(\mathbf{a}^\top [\mathbf{\Theta} h_i \,||\, \mathbf{\Theta} h_k]\right)\right)} \tag{6}$$

In the above equations, h'_i denotes the updated feature vector of node i, obtained by aggregating information from its neighbors, weighted by attention coefficients α_{ij}. The attention coefficients are computed using a LeakyReLU-activated dot product of the transformed features of nodes i and j, scaled by the transposed learnable weight vector $\mathbf{a}^\top$, and normalized using the exponential function $\exp$. The linear transformation of node features is facilitated by a learnable weight matrix $\mathbf{\Theta}$. ReLU is the default activation function applied to the output, introducing non-linearity to the model. These components collectively ensure that the model dynamically allocates varying levels of importance to different nodes within the neighborhood, thereby amplifying the relevance of critical nodes. This allows the model to focus on parts of the input graph with significant informational content.

2) GraphSAGE Layer: The GraphSAGE layer provides a methodology for generating embeddings by sampling and aggregating features from a node's local neighborhood. In our model, GraphSAGE is utilized in two layers:

$$h_v^{(k)} = \sigma\left(W \cdot AGGREGATE^{(k)}\left(h_u^{(k-1)} : u \in N(v)\right)\right) \tag{7}$$

where $h_v^{(k)}$ is the feature vector of node v at the k-th layer, $N(v)$ is the neighborhood of node v, W is a learnable weight matrix. $AGGREGATE$ is a mean function aggregating neighbor information into a single vector, and σ is an activation function. Integrating the GAT module with the GraphSAGE module dramatically enhances the model's perceptual ability towards various graph structures, effectively integrating node connectivity information into the node representation.

3) Integrated Decision Pooling Module: Existing common pooling methods include Graph Max Pooling, Graph Average Pooling, TopK Pooling, etc. In order to reduce computational complexity, these methods select node information, resulting in information loss. Flow interaction graphs have limited nodes, so we designed an Integrated Decision Pooling (IDP) module to consider information from every node. The module first performs feature extraction and transformation of all nodes through a multilayer perceptron (MLP) and then performs a global pooling operation to create a uniform graph embedding. Here are the IDP module details.

MLP Transformation: Given node features represented as $\mathcal{H}_l$, the MLP processes them through three layers as follows:

$$\mathcal{H}_{l+1} = \text{Dropout}\left(\text{PReLU}\left(\mathcal{W}_l \mathcal{H}_l\right)\right) \tag{8}$$

where $\mathcal{W}_l$ denotes the weight matrix of the l-th layer, and the transformation is applied iteratively for three layers.

Global Mean Pooling: After the MLP transformation, the node embeddings undergo the global mean pooling. This module aggregates individual node information to provide

a robust graph-level representation, enhancing its ability to handle graph-related tasks.

IV. Experiments

In this section, we conducted a series of experiments to demonstrate the applicability and performance of FlowMiner in various traffic classification tasks. Specifically, we aim to address the following questions: **RQ1:** How does FlowMiner perform in various tasks? **RQ2:** Is each newly proposed component effective? **RQ3:** How does hyperparameter variation affect FlowMiner? **RQ4:** What is the complexity of FlowMiner?

A. Experimental Settings

1) Dataset Setup: To evaluate the performance of FlowMiner, we selected eight public datasets from five encrypted traffic classification tasks to conduct experiments. Details for each task are as follows:

Task 1: Encrypted Application Classification. Utilizing the CSTNET dataset [3], we aim to classify 120 applications sourced from Alexa's Top-5000 using the TLS 1.3 protocol.

Task 2: VPN Traffic Classification. Employing the widely-used ISCX VPN-nonVPN dataset [10], we divide the data into the ISCX VPN and nonVPN datasets, both containing five categories.

Task 3: Tor Traffic Classification. With the ISCX Tor-nonTor dataset [11], we divide the data into the ISCX Tor and nonTor datasets, both containing eight categories, to evaluate classification performance in the realm of anonymous network.

Task 4: Censorship-resistant Proxy Traffic Classification. Utilizing data from the LFETT2021 dataset [12], we explore censorship-resistant proxy traffic classification in varied scenarios. The proxies used in this dataset include Vmess and ShadowsocksR (SSR) protocols. We divide the data into the LFETT2021-Vmess and LFETT2021-SSR datasets containing 71 and 65 classes, respectively.

Task 5: Encrypted Malicious Traffic Classification. Using the USTC-TFC dataset [13], with ten benign and ten malicious traffic categories, we aim to distinguish malicious traffic from benign traffic in the presence of encryption to identify and block malicious activities.

2) Data Preprocessing: In the data preprocessing phase, we first encoded the labels of the non-numeric type columns in the dataset into numeric type variables that the machine learning model can understand. Next, we employed a simple interpolation strategy to fill in missing values in numeric-type columns by using the average of each column. Finally, we applied a min-max normalization technique to scale all numeric-type data to the $[0, 1]$ range to reduce the model's bias in prediction due to differences in feature scales. In addition, due to the small number of samples in some datasets, we could not construct sufficient graphs for training and testing. So, we removed these classes. This series of preprocessing steps ensures complete, standardized, and consistent data. This provides a solid foundation for subsequent model training.

3) Implementation Details: In the training phase, the batch size is set to 64. We use Adam with a learning rate of 0.001. The maximum epoch is set to 500. All datasets are divided according to training, testing, and validation sets with a ratio of 8:1:1. Due to the data imbalance problem in some datasets, we adopt stratified sampling. Traffic feature extraction is implemented based on NFStream and DPKT libraries. Graph modeling is implemented based on torch geometric 2.0.4. Each experiment is independently repeated ten times, and the average result is reported.

4) Evaluation Metrics: We adopt four metrics for evaluation: Accuracy (AC), Recall (RE), Precision (PR), and F1 Score (F1), utilizing the macro-average method for the latter three. Definitions are as follows: Accuracy is the ratio of correctly predicted instances Accuracy $= \frac{\text{TP + TN}}{\text{TP + TN + FP + FN}}$; Precision, Precision $= \frac{\text{TP}}{\text{TP + FP}}$, and Recall, Recall $= \frac{\text{TP}}{\text{TP + FN}}$, denote the correctness and completeness of the positive predictions, respectively; F1 Score is the harmonic mean of Precision and Recall, F1-Score $= 2 \times \frac{\text{PR} \times \text{RE}}{\text{PR} + \text{RE}}$. The macro-average for each metric is calculated by averaging the metric independently computed for each class, Macro-average Metric $= \frac{1}{N} \sum_{i=1}^{N} \text{Metric}_i$, where N is the number of classes, ensuring equal contribution from all classes even in the presence of class imbalance.

B. Comparison Experiments (RQ1)

We compare different SOTA methods across multiple datasets, including (1) Machine Learning methods: CUMUL [15], BIND [14]; (2) Deep Learning methods: FS-Net [16], ET-Bert [3]; (3) Graph-based Deep Learning methods: GraphDApp [17] and TFE-GNN [2]. Additionally, given the excellent performance of tree-based models on tabular traffic feature data [18], we implemented three baselines based on the Random Forest for comparison. Among them, RF with all features (RF-AF) incorporates all the initial node features we proposed. Random Forest with traditional features (RF-TF) uses traditional spatio-temporal traffic features (including length, temporal, and content features). Random Forest with traditional and byte features (RF-TBF) utilizes traditional spatio-temporal traffic features and byte features. Also, machine learning-based methods all use LightGBM [19] to learn and classify extracted features. Note that FlowMiner completes the traffic graph classification task. All flow samples within a graph are labels of the same graph, so FlowMiner can be compared with flow-level traffic classification methods.

Table I and Table II show the experimental results. From the results, we draw the following conclusions. (1) **FlowMiner has achieved the best results in almost all tasks, demonstrating its exceptional generalization and robustness.**. Even if it is not the best result in some tasks, its result is very close to the optimal value. (2) FlowMiner's comprehensive ability and robustness are more prominent. Most of the models perform well on a few datasets. However, there is a significant drop in effectiveness on other datasets, such as FS-Net. This is because these models lack structural features in flow interaction graphs. (3) The baseline methods for ISCX

TABLE I: Comparison Results on ISCX VPN-nonVPN and ISCX Tor-nonTor Datasets

Dataset	ISCX VPN				ISCX nonVPN				ISCX Tor				ISCX nonTor			
Method	AC	PR	RE	F1	AC	PR	RE	F1	AC	PR	RE	F1	AC	PR	RE	F1
BIND [14]	0.9738	0.8745	0.8908	0.8756	0.8718	0.8745	0.8908	0.8756	0.7172	0.6131	0.5562	0.5655	0.9896	0.9643	**0.9860**	**0.9738**
CUMUL [15]	0.7098	0.6417	0.6531	0.6404	0.6593	0.5830	0.6047	0.5900	0.1883	0.1441	0.1072	0.1104	0.8654	0.6477	0.6058	0.6216
FS-Net [16]	0.9373	0.9250	0.9333	0.9268	0.8056	0.8080	0.8110	0.8037	0.5800	0.5084	0.4505	0.4534	0.9588	0.8300	0.8443	0.8341
ET-Bert [3]	0.9700	0.9800	0.9740	0.976	0.8318	0.8450	0.8480	0.843	0.8182	0.8960	0.7500	0.683	0.7852	0.786	0.782	0.7710
GraphDApp [17]	0.4223	0.4139	0.4162	0.4016	0.3347	0.2636	0.2788	0.2474	0.3556	0.3829	0.3959	0.3637	0.3052	0.3234	0.3340	0.3183
TFE-GNN [2]	0.9286	0.9008	0.9286	0.9048	0.9333	0.9000	0.9250	0.8827	0.8889	0.8426	0.8444	0.7963	0.9444	0.8889	0.9167	0.8611
RF-TF	0.9870	0.9726	0.9754	0.9738	0.6628	0.8003	0.7244	0.7225	0.8419	0.8512	0.7815	0.7965	0.9600	0.8379	0.7726	0.7992
RF-TBF	0.9896	0.9791	0.9802	0.9795	0.7557	0.8624	0.8018	0.8160	0.9041	0.9271	0.8732	0.8861	0.9805	0.9496	0.8459	0.8871
RF-AF(Our Features)	**0.9951**	0.9842	0.9887	0.9864	0.9883	0.9660	0.9411	0.9522	0.9446	0.9432	0.8990	0.9108	**0.9956**	0.9765	0.9554	0.9649
FlowMiner	0.9857	**0.9875**	**0.9917**	**0.9867**	**0.9985**	**0.9991**	**0.9983**	**0.9987**	**0.9800**	**0.9700**	**0.9800**	**0.9733**	0.9900	**0.9900**	0.9800	0.9733

TABLE II: Comparison Results on LFETT2021, CSTNET-TLS 1.3 and USTC-TFC Datasets

Dataset	LFETT2021-Vmess				LFETT2021-SSR				CSTNET-TLS 1.3				USTC-TFC			
Method	AC	PR	RE	F1	AC	PR	RE	F1	AC	PR	RE	F1	AC	PR	RE	F1
BIND [14]	0.9027	0.8871	0.9160	0.8979	0.8968	0.9063	**0.9283**	0.9138	0.9897	0.9805	0.9840	0.9813	0.6406	0.8968	0.9290	0.8913
CUMUL [15]	0.4911	0.4467	0.3959	0.4040	0.3277	0.3135	0.2838	0.2853	0.5389	0.5018	0.4785	0.4698	0.6480	0.6911	0.5555	0.5681
FS-Net [16]	0.3797	0.2737	0.2945	0.2500	0.3124	0.2186	0.2522	0.2019	0.7584	0.6886	0.6886	0.6552	0.5858	0.6350	0.6453	0.6163
ET-Bert [3]	0.7307	0.7330	0.7190	0.7290	0.7629	0.7780	0.7560	0.7580	0.8915	0.8890	0.8750	0.8780	0.9866	0.9960	0.9860	0.9910
GraphDApp [17]	0.2304	0.2173	0.2153	0.2134	0.1325	0.1200	0.1190	0.1166	0.2143	0.2105	0.2092	0.2071	0.3872	0.3501	0.3743	0.3318
TFE-GNN [2]	0.6667	0.4261	0.4187	0.4046	0.5833	0.3112	0.2692	0.2789	0.7778	0.5914	0.5863	0.5593	0.8444	0.7448	0.7650	0.7178
RF-TF	0.6247	0.6296	0.5738	0.5889	0.5946	0.5938	0.5343	0.5521	0.8031	0.7968	0.7774	0.7965	0.8659	0.9070	0.8710	0.8760
RF-TBF	0.7840	0.8045	0.7322	0.7567	0.7456	0.7934	0.7149	0.7420	0.9456	0.9466	0.9376	0.9407	0.9144	0.9360	0.9061	0.8871
RF-AF(Our Features)	0.7004	0.7350	0.6568	0.6792	0.6836	0.7087	0.6280	0.6530	**0.9945**	**0.9938**	**0.9924**	**0.9929**	0.9984	0.9980	0.9975	0.9977
FlowMiner	**0.9645**	**0.9632**	**0.9621**	**0.9573**	**0.9378**	**0.9348**	0.9227	**0.9187**	0.9875	0.9834	0.9832	0.9815	**0.9999**	**0.9995**	**0.9995**	**0.9995**

TABLE III: Ablation Study on ISCX VPN and ISCX Tor Datasets (Top: ISCX VPN, Bottom: ISCX Tor)

Method	Accuracy	Precision	Recall	F1 Score
w/o CF	0.9684 0.9400	0.9633 0.9092	0.9880 0.9117	0.9709 0.9067
w/o BF	0.9421 0.7077	0.9247 0.6412	0.9580 0.7750	0.9342 0.6885
w/o IDP	0.9684 0.9200	0.9633 0.8833	0.9880 0.8833	0.9709 0.8833
FlowMiner	**0.9857** **0.9800**	**0.9875** **0.9833**	**0.9917** **0.9833**	**0.9867** **0.9667**

Tor and LFETT2021-Vmess datasets are generally poor. A key reason for this is that their disguising and obfuscating properties significantly complicate traffic classification. Due to a robust flow association analysis mechanism, FlowMiner achieved superior results in these three more complex datasets (**F1 score improves by 5.94% to 86.29% compared to other models**). This provides better encoding capability for confusing and disguised traffic. (4) VPN does not employ disguise and confusion mechanisms similar to the censorship-resistant proxy. So, from the comparison results for the ISCX VPN and nonVPN datasets, VPN traffic may actually have even more pronounced features than regular traffic. (5) Consistent with previous research conclusions [18], tree-based models still have advantages over tabular traffic data. This can be seen in the experimental results of the Random Forest and the BIND model. We found that tree-based models perform well when extracted statistical features are effective.

C. Ablation Study (RQ2)

Due to space constraints, we conducted ablation experiments on FlowMiner using the ISCX VPN and Tor datasets to assess the contribution of the proposed modules to the model. The FlowMiner model was ablated for Byte Features (BF), Cross Features (CF), and Integrated Decision Pooling module (IDP) in separate experiments. From Table III, we can see that in different datasets, the results of eliminating the three modules are as follows: the F1 score decreased by 1.58%-6.00%, 5.25%-27.82%, and 1.58%-8.34% after eliminating CF, BF and IDP, respectively. This indicates that byte features play a more critical role in classification. The CF and IDP modules also contribute significantly to FlowMiner's performance.

We further tested by replacing the GNN model structure (GAT [20], GraphSAGE [21], GCN [22] and SGC [23]) while keeping the input features unchanged. Figure 4 shows the experimental results. Although all models have indicated some classification capabilities, FlowMiner has exhibited a test accuracy of over 98% on both datasets, which is significantly better than other models, proving its superiority. Specifically, on the ISCX VPN dataset, the F1 score of FlowMiner has improved by 0.6%, 0.38%, 2.89%, and 0.83% compared to other GNN models. On the ISCX Tor dataset, the F1 score improvement of the FlowMiner model is even more significant, increasing by 8.96%, 4.29%, 6.57%, and 10.60% compared to the other GNN models, respectively. In summary, the combination of different GNN architectures effectively enhances the traffic representation capability of our model.

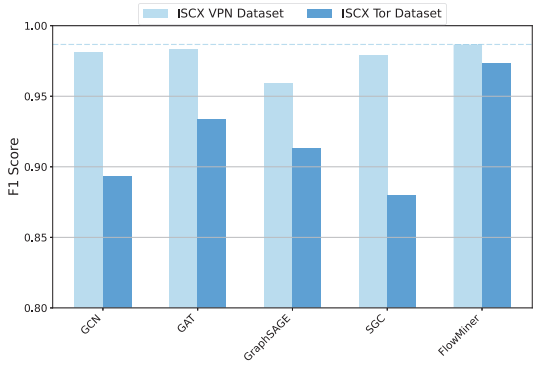

Fig. 4: GNN Architecture Ablation Analysis.

Fig. 5: The Impact of Maximum Expected Node Number on FlowMiner.

D. Model Sensitivity Analysis (RQ3)

The maximum expected node number determines the number of nodes in the generated graph and is a major hyperparameter in FlowMiner. To study the impact of the maximum expected node number on our model, we conducted sensitivity experiments on the ISCX VPN and Tor datasets. Figure 5 shows the model's relative stability for the ISCX Tor dataset. All evaluation metrics exhibit minor fluctuations when the number of nodes increases from 100 to 1000. However, a more significant performance fluctuation trend is observed for the VPN dataset. This may be attributed to the fact that within the VPN dataset, as the node scale expands, traffic distribution or network behavior becomes more diversified and irregular, thereby increasing the complexity of classification or analysis.

Overall, the impact of the maximum expected node number on the model's performance decline is limited (with a decreased range of 4.00%-7.25% on all metrics). We chose a node number 500 as our default setting to balance model performance and computational efficiency. This setup offers a buffer to resist data noise and fluctuations, ensuring model stability and generalization capability. This parameter can be further modified in practical model applications according to dataset size.

E. Model Complexity Analysis (RQ4)

To evaluate the complexity of FlowMiner, we calculate the floating-point operations (FLOPs) and model size of all deep learning models. Simultaneously, we show the average F1 scores of the model across all datasets. From the comparative results in Table IV, we can conclude that FlowMiner not only achieves higher precision but also manifests a clear advantage in reducing costs for training and inference. As FlowMiner employs GNN for learning, it essentially learns the weight coefficients of node interactions. It provides a particular advantage over other models regarding data volume. In contrast, although GraphDApp is highly lightweight, its performance is significantly lower than other models due to its suboptimal graph construction method.

Notably, unlike packet counts, the number of flows does not increase linearly with the size of traffic data. This implies that FlowMiner possesses notable scalability advantages during operation in large-scale network environments. We can easily scale FlowMiner's data processing by adjusting the maximum expected node number. This enables FlowMiner to adapt flexibly to varying-scale network environments, demonstrating substantial practical value and application potential.

TABLE IV: Model FLOPs and Parameters

Model	FLOPs(M)	Parameters(M)	Average F1 Score
FS-Net[16]	1.0e+2	3.2e+0	0.5927
ET-BERT[3]	1.1e+4	8.6e+1	0.8286
GraphDApp[17]	3.8e-2	1.1e-2	0.2750
TFE-GNN[2]	2.2e+3	4.4e+1	0.6757
FlowMiner	1.8e+1	1.4e-03	0.9736

V. Real-World Evaluation

We designed this experiment to evaluate FlowMiner's effectiveness. However, assessing encrypted traffic classification tasks in the real world is challenging, particularly in finding the ground truth. In this study, we chose the malicious traffic classification task and used the Emerging Threat Ruleset (ET Ruleset), maintained by experts, as the ground truth for evaluation.

To replicate real-world evaluation as closely as possible, we collaborated with an ISP vendor to conduct tests at a traffic egress node with a capacity of 40Gbps. We first manually downloaded malicious traffic by searching for APT keywords in ANY.RUN sandbox and selected actual malicious sample traffic from the malware-traffic-net website from 2013 to 2023. These were used as malicious traffic, with background traffic filtered out using the ET Ruleset. The ISP egress traffic was combined as white traffic to form the first test scenario, RealWorld1. Next, we deployed and configured the Suricata engine with the ET Ruleset to obtain malicious traffic. We combined it with ISP egress background traffic as white traffic to construct the second test scenario, RealWorld2. RealWorld1 contains 1,354,560 flows, with 284,450 malicious. RealWorld2 contains 1,093,884 flows, with 23,774 malicious. We used 1% of each dataset for training, resulting in a 1:99 training-to-testing ratio in both scenarios. The comparison model is BIND, which performed the best among all baseline models in our comparative experiments. The results show that BIND achieved Recall, Precision, and F1 Scores of 87.57%, 97.59%, and 91.75% on RealWorld1, and 50.00%, 49.88%, and 49.94% on RealWorld2. FlowMiner, on the other hand, achieved

Recall, Precision, and F1 Scores of 98.56%, 99.16%, and 98.85% on RealWorld1, and 82.40%, 95.70%, and 87.87% on RealWorld2.

The results show that the model's performance in RealWorld2 is significantly lower than in RealWorld1. This is because the malicious traffic in RealWorld2 comes from real-world scenarios where benign traffic distribution is highly random. This randomness can cause benign and malicious traffic to exhibit similar behaviors, making them harder to distinguish. In contrast, the malicious and benign traffic in RealWorld1 has different sources, making classification easier for the model. **This issue may be a key reason why many SOTA traffic classification models fail in real-world applications. Despite this, FlowMiner maintains over 95% classification precision in both real-world evaluation tasks, demonstrating its strong generalization and robustness through flow association mining.**

To better understand why FlowMiner is effective, we performed a case analysis using the explanatory capabilities of the Emerging Threat Ruleset for malicious behaviors. Note that the same rule alert may be triggered by multiple flows but is consolidated in the case presentation.

Attack Case 1: The attacker uses the Log4j vulnerability to control the system (Alert ID: 2034674), implants WaterTiger malware (Alert ID: 2821183), and exfiltrates data with RisePro malware (Alert ID: 2034674).

Attack Case 2: The attacker gains database info via SQL injection (Alert IDs: 2006445, 2011042, 2017807, 2017808), executes remote code with PHP/Base64 commands (Alert IDs: 2025801, 2025806), and establishes a reverse shell to control the server (Alert ID: 2044751).

From these cases, it is evident that FlowMiner can automatically aggregate malicious flows with multiple associations into flow interaction graphs and assess their overall maliciousness through graph relationship mining and graph neural networks. This significantly enhances the accuracy of malicious threat detection. **Notably, unlike the ET Ruleset, which requires expert knowledge for maintenance, FlowMiner can construct flow interaction graphs and conduct comprehensive assessments with high accuracy without needing step-by-step attack detection for each flow.**

VI. Related Works

Deep Learning Methods: Classical machine learning methods often rely on expert experience to filter traffic features [24]. With the continuous development of deep learning, many scholars have utilized its powerful automatic feature extraction capabilities to address the the above problem [25]. For example, Xie et al. [26] extracted the header bytes of traffic packets and realized accurate traffic classification through the self-attention mechanism. Xie et al. [1] proposed a spatio-temporal sequence feature model for modeling traffic features. Liu et al. [16] proposed an encoder-decoder model to learn the traffic feature representation through a reconstruction mechanism. Lin et al. [3] developed ET-Bert, a Bert-based traffic pre-training model, which pre-trains burst-level traffic masks and performs classification through fine-tuning in downstream tasks. However, existing deep learning methods cannot realize correlation analysis between flows due to the limitation of single-level analysis and often only perform well on specific tasks.

Graph Deep Learning Methods: As an effective method for processing graph data, graph neural networks have been gradually applied to various cybersecurity problems. Pham et al. [6] only used temporal correlation attributes between flow to construct a traffic graph. This is easily affected by concurrent flow noise. Shen et al. [17] constructed a graph structure called the Traffic Interaction Graph based on traffic packet lengths. Zhang et al. [2] constructed traffic graphs by correlating traffic bytes. In summary, these methods only utilize partial traffic information and lack a thorough exploration of interaction information across flows between hosts. This limits their adaptability to different tasks.

To enhance the ability of existing models to mine association relationships between flow samples, we propose FlowMiner. It elegantly integrates heterogeneous features from different dimensions using GNN and accurately identifies traffic by mining potential patterns in the graph feature space.

VII. Discussion

This section discusses the limitations of our work. While we examine interactions between flow samples, packet behavior within a flow is still represented by statistical features, causing some information loss. GNN, with its powerful representation learning, could replace statistical methods to capture more precise packet length distribution patterns.

Secondly, FlowMiner employs a homogeneous graph classification method to characterize all nodes as the same type. For cross-level correlation analysis (e.g., between packet-level nodes and flow-level nodes), it may be necessary to model them through heterogeneous graph classification techniques.

Finally, FlowMiner employs supervised training in this paper. In the real world, it is feasible to obtain large amounts of unsupervised traffic data. It is also worth exploring whether this data can be used to further improve FlowMiner's performance by combining graph pre-training methods [27].

VIII. Conclusion

This paper introduces FlowMiner, an encrypted traffic classification model leveraging flow relation mining. By extracting multidimensional flow features, constructing interaction graphs, and learning via GNN, FlowMiner outperforms SOTA methods in effectiveness and robustness across multiple tasks and shows strong generality in real-world evaluations.

Acknowledgment

This work is supported by the Scaling Program of Institute of Information Engineering, CAS (Grant No. E3Z0041101).

Conflict of Interest

The authors have no competing interests to declare that are relevant to the content of this article.

References

[1] J. Xie, S. Li, X.-c. Yun, Y. Zhang, and P. Chang, "Hstf-model: An http-based trojan detection model via the hierarchical spatio-temporal features of traffics," *Comput. Secur.*, vol. 96, p. 101 923, 2020.

[2] H. Zhang, L. Yu, X. Xiao, *et al.*, "Tfe-gnn: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," *Proceedings of the ACM Web Conference 2023*, 2023.

[3] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *The Web Conference*, 2022, pp. 633–642.

[4] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on tabular data?" *ArXiv*, vol. abs/2207.08815, 2022.

[5] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" In *International Conference on Learning Representations*, 2019.

[6] T.-D. Pham, T.-L. Ho, T. T. Huu, T.-D. Cao, and H. L. Truong, "Mappgraph: Mobile-app classification on encrypted network traffic using deep graph convolution neural networks," *Annual Computer Security Applications Conference*, 2021.

[7] M. Jiang, Z. Li, P. Fu, *et al.*, "Accurate mobile-app fingerprinting using flow-level relationship with graph neural networks," *Comput. Networks*, vol. 217, p. 109 309, 2022.

[8] M.-L. Wu, J. Sippe, D. Sivakumar, *et al.*, "How the great firewall of china detects and blocks fully encrypted traffic," in *USENIX Security Symposium*, 2023.

[9] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *IEEE International Conference on Computer Vision*, 2015, pp. 1026–1034.

[10] G. D. Gil, A. H. Lashkari, M. Mamun, and A. A. Ghorbani, "Characterization of encrypted and vpn traffic using time-related features," in *International Conference on Information Systems Security and Privacy*, 2016, pp. 407–414.

[11] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of tor traffic using time based features," in *International Conference on Information System Security and Privacy*, 2017, pp. 253–262.

[12] Z. Gu, G. Gou, C. Hou, G. Xiong, and Z. Li, "Lfett2021: A large-scale fine-grained encrypted tunnel traffic dataset," *2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pp. 240–249, 2021.

[13] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," *2017 International Conference on Information Networking (ICOIN)*, pp. 712–717, 2017.

[14] K. Al-Naami, S. Chandra, A. M. Mustafa, *et al.*, "Adaptive encrypted traffic fingerprinting with bi-directional dependence," *Proceedings of the 32nd Annual Conference on Computer Security Applications*, 2016.

[15] A. Panchenko, F. Lanze, A. Zinnen, *et al.*, "Website fingerprinting at internet scale," in *Annual Network and Distributed System Security Symposium*, 2016.

[16] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "Fs-net: A flow sequence network for encrypted traffic classification," *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications*, pp. 1171–1179, 2019.

[17] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 2367–2380, 2021.

[18] A. Lichy, O. Bader, R. Dubin, A. Dvir, and C. Hajaj, "When a rf beats a cnn and gru, together - a comparison of deep learning and classical machine learning approaches for encrypted malware traffic classification," *Comput. Secur.*, vol. 124, p. 103 000, 2022.

[19] G. Ke, Q. Meng, T. Finley, *et al.*, "Lightgbm: A highly efficient gradient boosting decision tree," in *Neural Information Processing Systems*, 2017.

[20] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations*, 2018.

[21] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Conference on Neural Information Processing Systems*, 2017.

[22] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations*, 2017.

[23] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *International Conference on Machine Learning*, 2019, pp. 6861–6871.

[24] T. van Ede, R. Bortolameotti, A. Continella, *et al.*, "Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," *Proceedings 2020 Network and Distributed System Security Symposium*, 2020.

[25] P. Sirinam, M. Imani, M. Juárez, and M. K. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018.

[26] G. Xie, Q. Li, and Y. Jiang, "Self-attentive deep learning method for online traffic classification and its interpretability," *Computer Networks*, vol. 196, p. 108 267, 2021.

[27] Y. Lu, X. Jiang, Y. Fang, and C. Shi, "Learning to pre-train graph neural networks," in *AAAI Conference on Artificial Intelligence*, 2021.